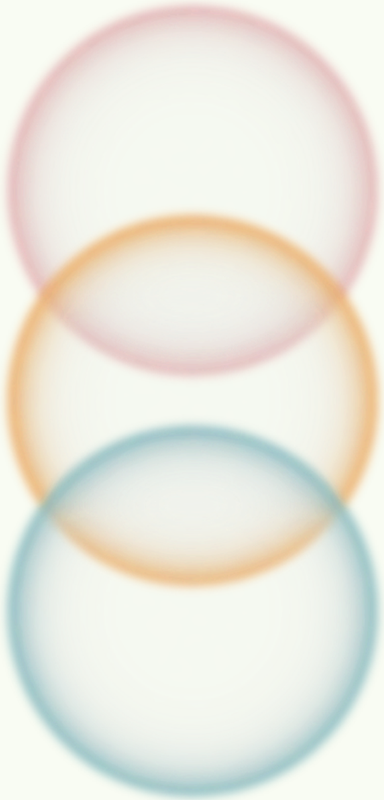
Diseño

Instructor Manuel Porras
mf.porras@uniandes.edu.co

Miremos algo importante



¿Qué es una enumeración ?

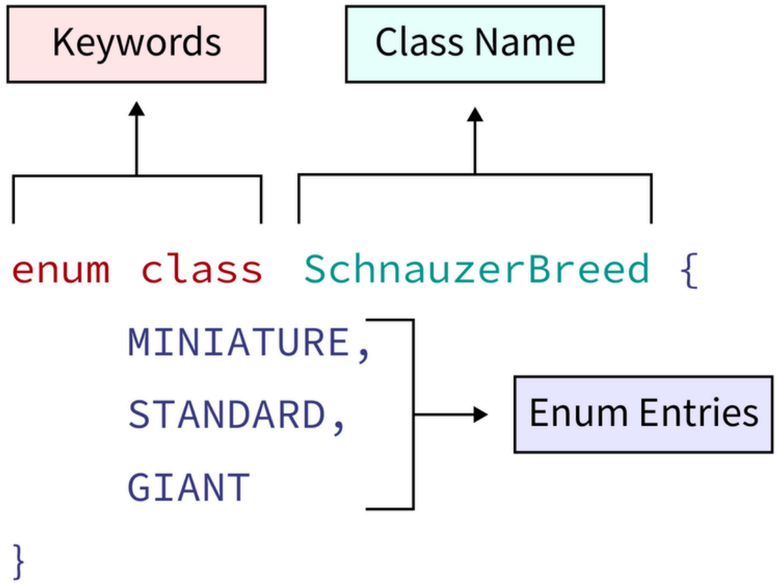
Las enumeraciones en Java

Son la manera en que implementamos ciertas reglas del dominio, ya que limitan la creación de objetos.

¿Qué valores especificos puede tener un atributo?

¿Me limito a tener ciertas opciones?

¿Cuál es la diferencia con la constante de una clase?



SCALER Topics

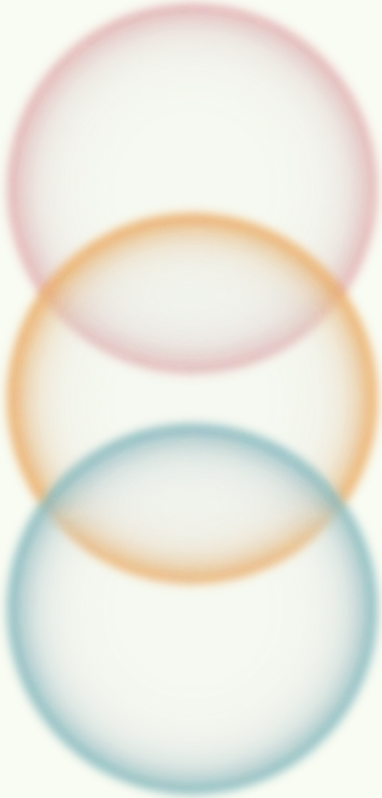
Enum vs Class

```
e.g.:
public enum Season{SPRING, SUMMER, FALL, WINTER}

public class Season2
{
    public static final int SPRING = 0;
    public static final int SUMMER = 1;
    public static final int FALL = 2;
```

Java constants are spelled in all upper-case letters

¿Qué es diseñar?



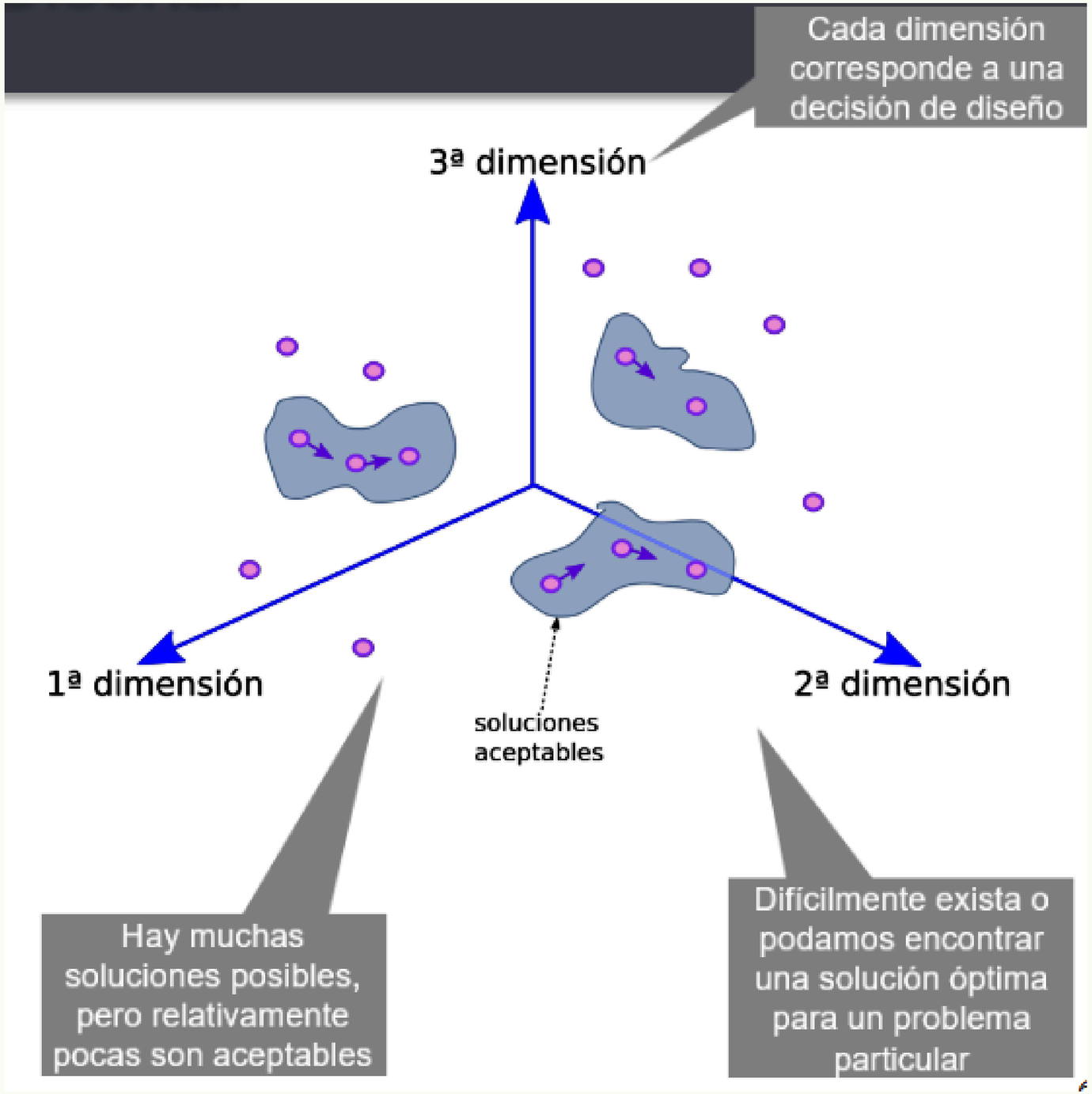
Lo que hicieron del proyecto y taller 4, ¿es la única solución posible?

¿Qué es diseñar?

- Diseñar es **tomar decisiones.**
- Diseñar es el proceso para encontrar y seleccionar una **opción aceptable** dentro del espacio de las soluciones para un problema.

¿Qué es diseñar?

- Diseñar es **construir** abstracciones de información y de operaciones
- **Organizar** esas abstracciones en piezas de software funcionales.


Cada dimensión corresponde a una decisión de diseño
3ª dimensión
1ª dimensión
2ª dimensión
soluciones aceptables
Hay muchas soluciones posibles, pero relativamente pocas son aceptables
Difícilmente exista o podamos encontrar una solución óptima para un problema particular

La búsqueda de soluciones de un problema tiene ciertos elementos

Búsqueda de soluciones

Un contexto de diseño es un conjunto de requerimientos y restricciones en un dominio, dentro del cual tenemos que encontrar una solución.

Iterativo:
Se repite el proceso varias veces

Incremental:
Se va refinando progresivamente.

Basado en heuristicas:
Hay unas guías o atajos que nos ayudan a tener un buen diseño.

¿Qué buscamos en un buen diseño de software?

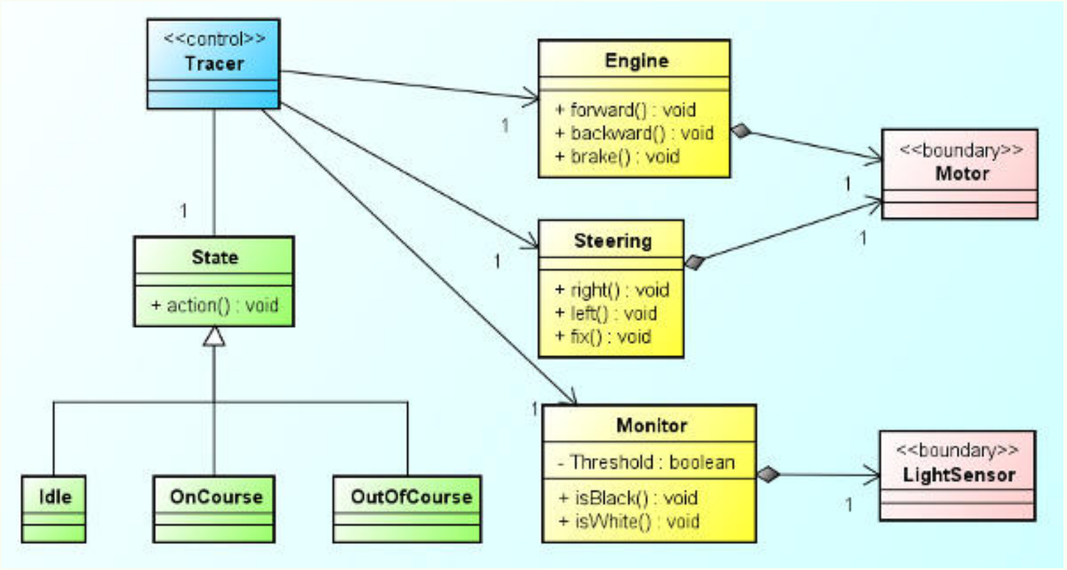
Entendamos primero estos dos conceptos

Cohesión

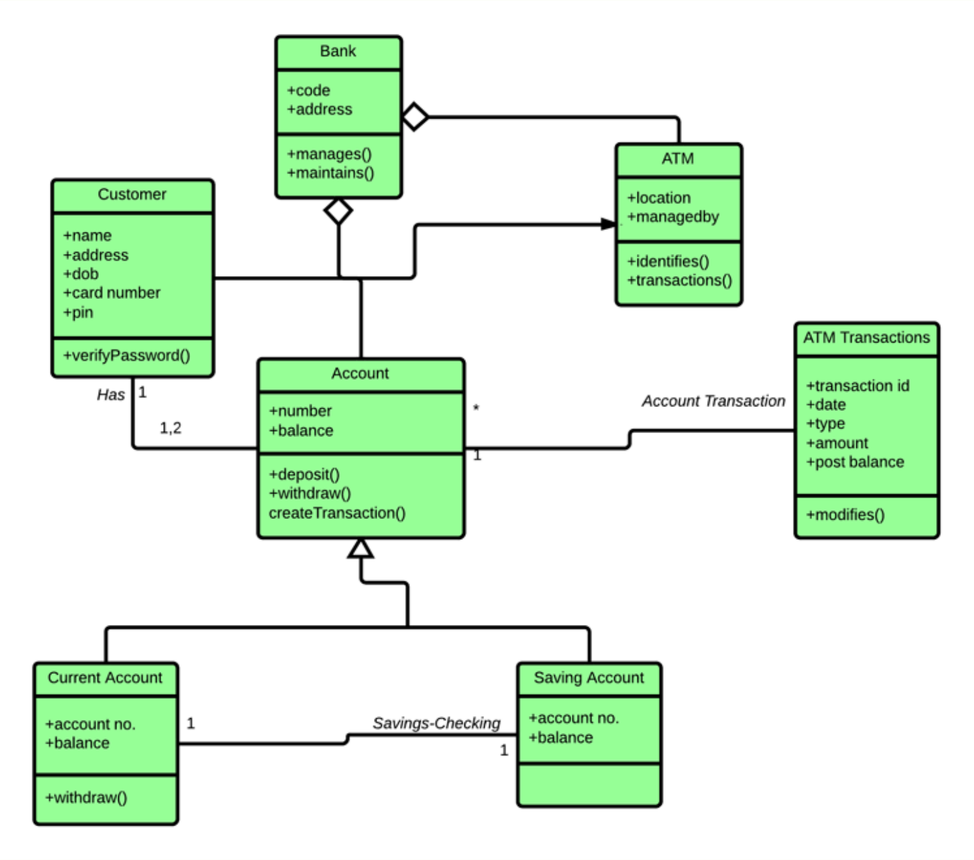
Acoplamiento

Cohesión

Se refiere al grado en que los elementos de un módulo permanecen juntos.

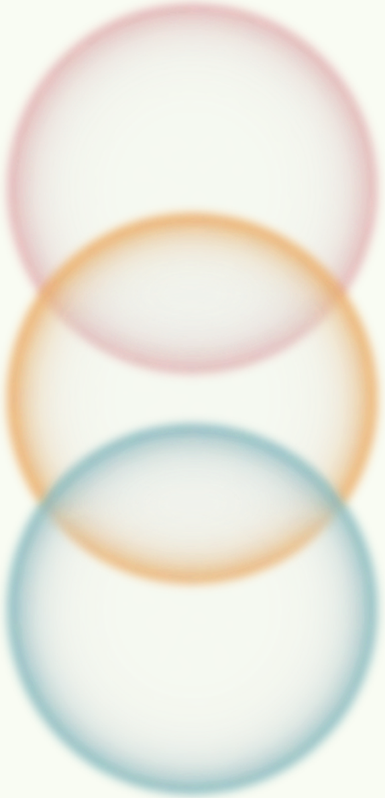


Acoplamiento



Se refiere al nivel de relación entre módulos de un sistema.

¿Qué nivel de cohesión y acoplamiento buscamos?



Buscamos bajo acoplamiento y alta cohesión.

Veamos un ejemplo de diseño

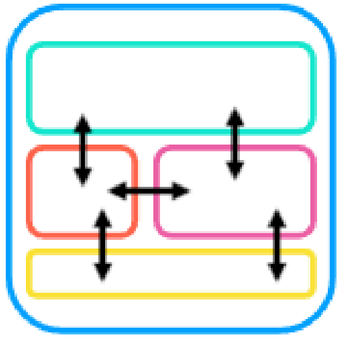
Diseño iterativo, incremental por niveles

Nivel 1: Sistema completo

Este es el sistema completo y se comprometió a cumplir con los RF, RNF y restricciones para la solución.

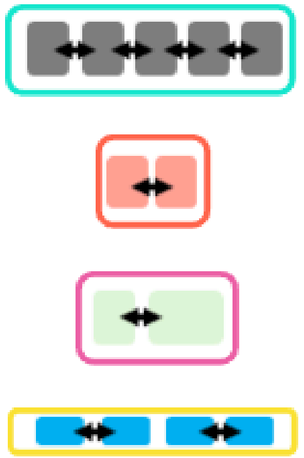
No sabemos cómo es por dentro, pero conocemos su frontera (qué hace y cómo se ve desde afuera)

Nivel 2: Primera descomposición



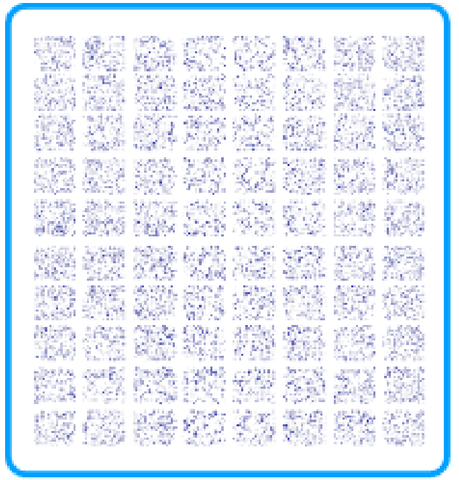
El sistema completo sigue cumpliendo con RF, RNF y restricciones. Pero ahora sabemos que por dentro hay 4 componentes. La suma de esos componentes le permite al sistema cumplir con sus compromisos.
De cada componente interno conocemos su frontera (qué hace, cómo se ve y cómo interactúa con los otros componentes) pero no sabemos cómo es por dentro.

Nivel 3: Segunda descomposición



Para cada componente repetimos el proceso: lo descomponemos para encontrar qué hay por dentro que nos ayude a cumplir con los requerimientos a los que se comprometió.
Para cada componente interno definimos una frontera en términos de los que hace y cómo se ve, y definimos la interacción entre estos componentes.

Nivel n: descomposición final



El proceso se repite hasta que los componentes no puedan descomponerse más (sean elementos implementables - usualmente serán clases).

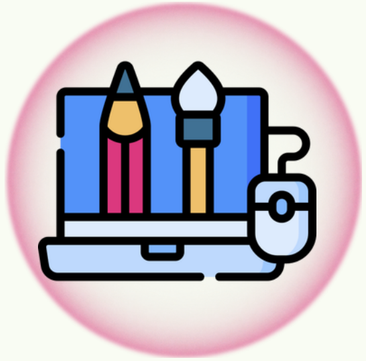
Ventajas del diseño iterativo e incremental

Simplificación:
No tenemos que pensar todo el tiempo en todos los detalles.

Abstracción:
Diseñar en términos de fronteras bien definidas es mucho más sencillo.

Distribución del trabajo:
Podemos distribuir naturalmente el trabajo entre los miembros de un equipo.

Desventajas del diseño iterativo e incremental



Raramente esta aproximación va a solucionar todos los problemas en una sola iteración.

Diseñar componentes totalmente por separado puede llevar a diseños muy diferentes.

Rara vez es evidente cómo hacer la descomposición de cada componente.

¡Diseñar es tomar decisiones!

Diseñar es tomar decisiones

Muchas de las decisiones que tomemos durante el proceso de diseño no son fáciles:

- No necesariamente tendremos claras todas las alternativas
- No necesariamente tendremos claros los criterios para tomar la decisión
- No necesariamente podremos saber si acertamos en la decisión

Todo es un TRADE-OFF

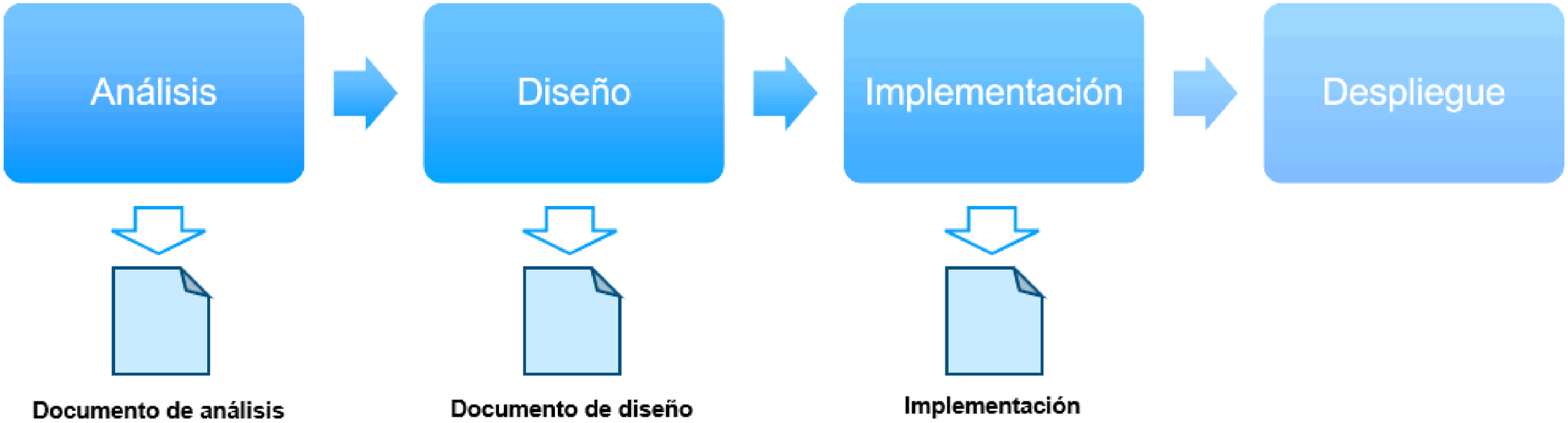
Cada decisión que tomemos encierra un compromiso entre lo que ganamos y lo que perdemos (o dejamos de ganar).

Pero, ¿Dónde ponemos el diseño formalmente?

El documento de diseño

Cuando nos referimos a **EL DISEÑO** de una aplicación, estamos hablando de una colección de decisiones.

El documento de diseño es el artefacto en el que quedan registradas las decisiones que se tomaron (ventajas, desventajas, alternativas consideradas, …)



¿De dónde sale el diseño?

¿De dónde sale el diseño?

A pesar de lo que puede parecer a veces, el diseño no es un proceso totalmente creativo donde la inspiración juegue el papel principal.

Por el contrario, es imposible diseñar sin tener en cuenta estos 4 elementos:

- Análisis
- Principios de diseño
- Experiencias previas
- Patrones

Elementos del análisis

Los elementos del análisis deben ser la base para hacer el diseño de una solución.

- Entendimiento del mundo del problema
  - Modelo del mundo del problema
  - Contexto
  - Reglas del dominio
- Requerimientos funcionales (RF) de la solución
  - Casos de uso, historias de usuario, etc.
- Requerimientos **no** funcionales (RNF) de la solución
  - Escenarios de calidad, etc.
- **Restricciones** del proyecto y de la operación de la solución

Experiencias previas

Las experiencias previas son claves para construir cada nuevo diseño.

Un nuevo problema muchas veces incluye retos a los que nos habíamos enfrentado en oportunidades anteriores.

El reto es reconocer que un nuevo problema es realmente un problema anterior con una nueva apariencia.

¿Y si no tenemos experiencia?
Toca investigar y leer

Patrones/Antipatrones

Los catálogos de patrones (y anti-patrones) son un excelente recurso para acumular el conocimiento que la experiencia no nos haya dado.

Patrones: Soluciones conocidas a problemas recurrentes

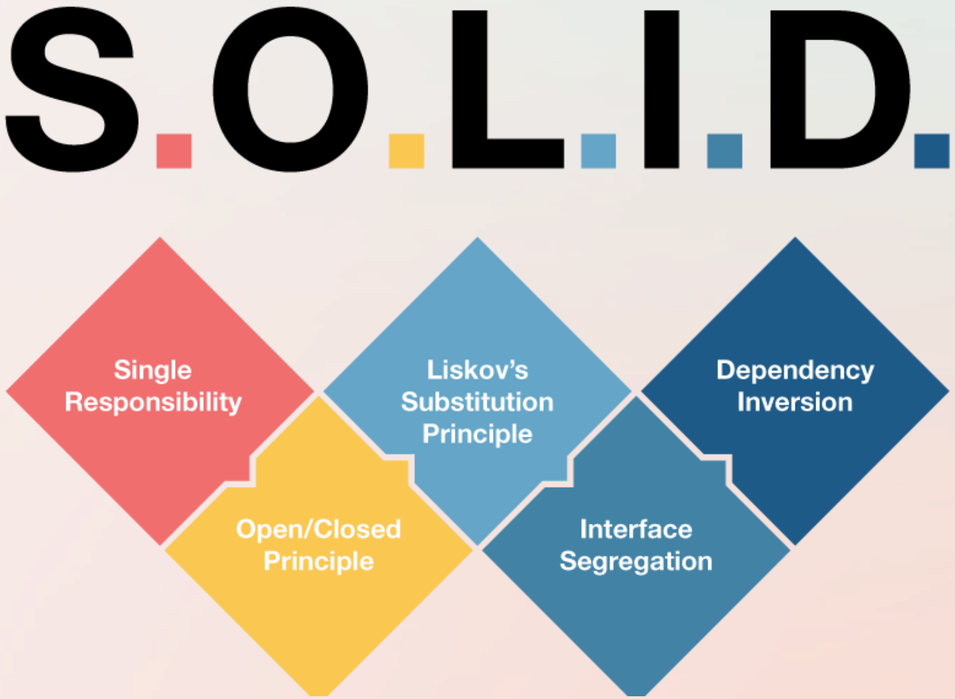
Anti-patrones: Soluciones problemáticas para problemas recurrentes.

Principios de diseño

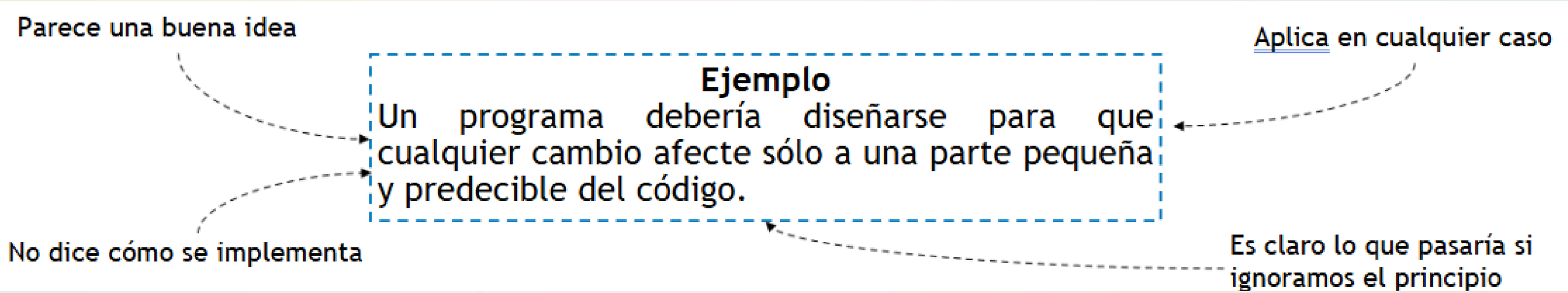
Un principio de diseño es una idea o herramienta básica que puede ser aplicada en una gran cantidad de contextos para lograr que el código sea más flexible, extensible o fácil de mantener.

Los principios de diseño no son reglas y son tan abstractos que se pueden implementar de muchas maneras diferentes.

Deberían tenerse en cuenta cuando se tomen decisiones: ignorarlos podría tener consecuencias negativas conocidas.



Un ejemplo de lo anterior


Parece una buena idea
Aplica en cualquier caso
Ejemplo
Un programa debería diseñarse para que cualquier cambio afecte sólo a una parte pequeña y predecible del código.
No dice cómo se implementa
Es claro lo que pasaría si ignoramos el principio

Veamos un ejemplo aplicado

Contenedor de aplicaciones Web

Contenedor de aplicaciones web

Queremos construir un Contenedor de Aplicaciones Web que va a permitirle a varias aplicaciones responder a peticiones HTTP (sólo GET, con parámetros).

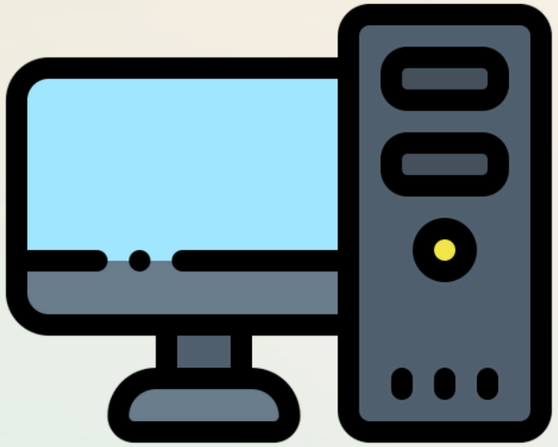
Contenedor de aplicaciones web

Queremos construir un Contenedor de Aplicaciones Web que va a permitirle a varias aplicaciones responder a peticiones HTTP (sólo GET, con parámetros).

¿Qué es eso de peticiones HTTP?

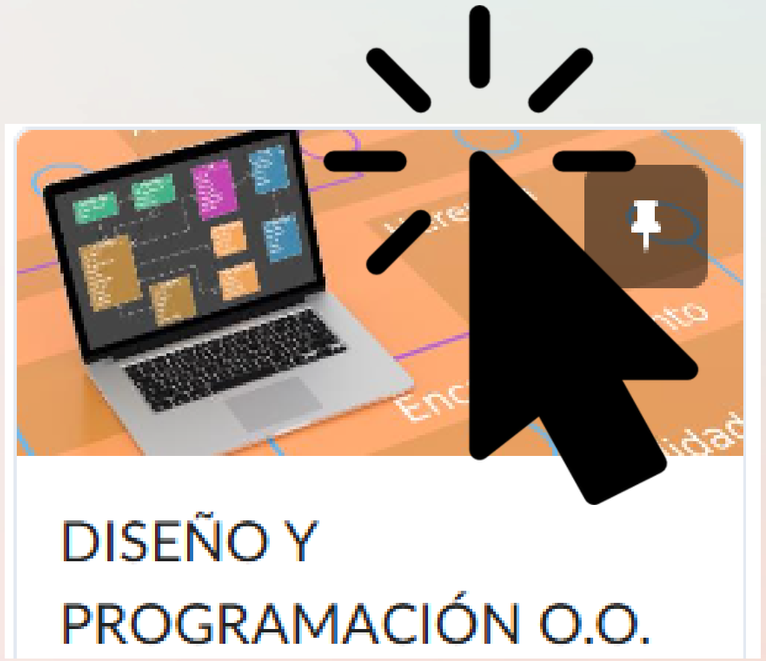
Primero entendamos como funcionan las peticiones Web y las API

Peticiones Web



Estudiante de DPOO

https://bloqueneon.uniandes.edu.co/d2l/home

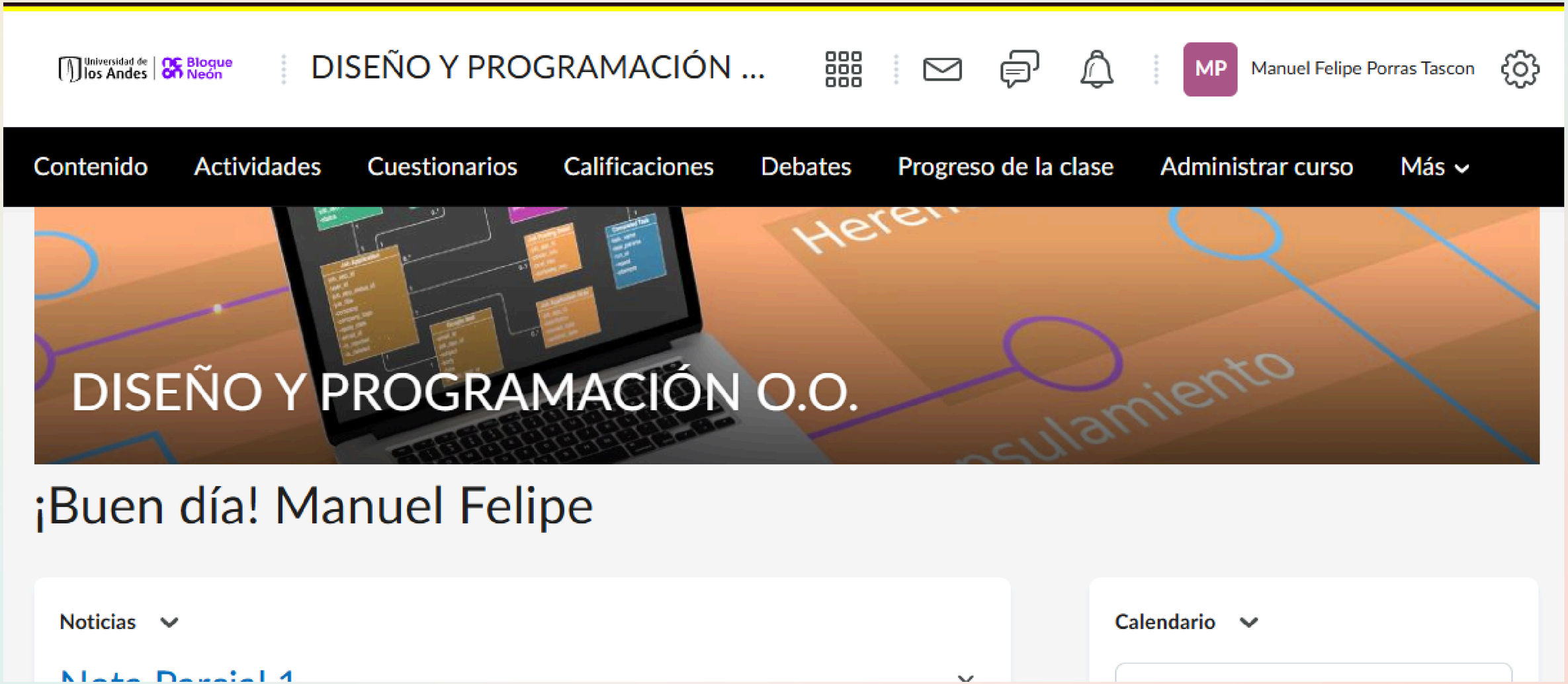


El estudiante entra al curso para estudiar juicioso

¿Qué creen que ocurre a nivel de página web?

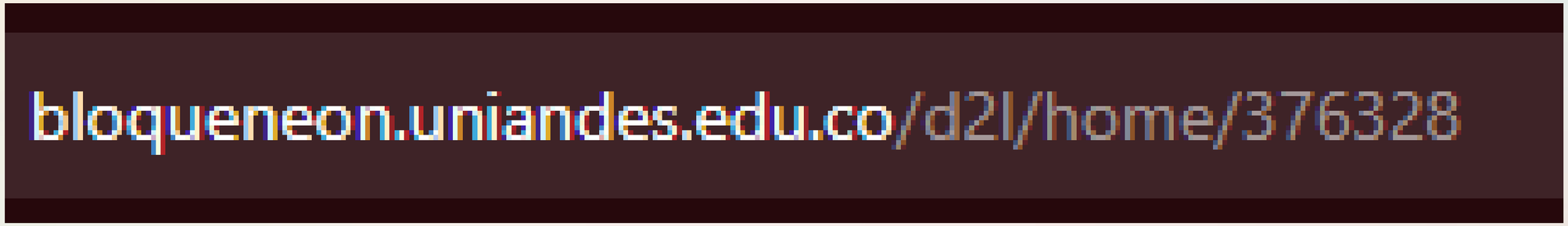
Peticiones Web

Pues a nivel visual vemos lo siguiente:



Peticiones Web

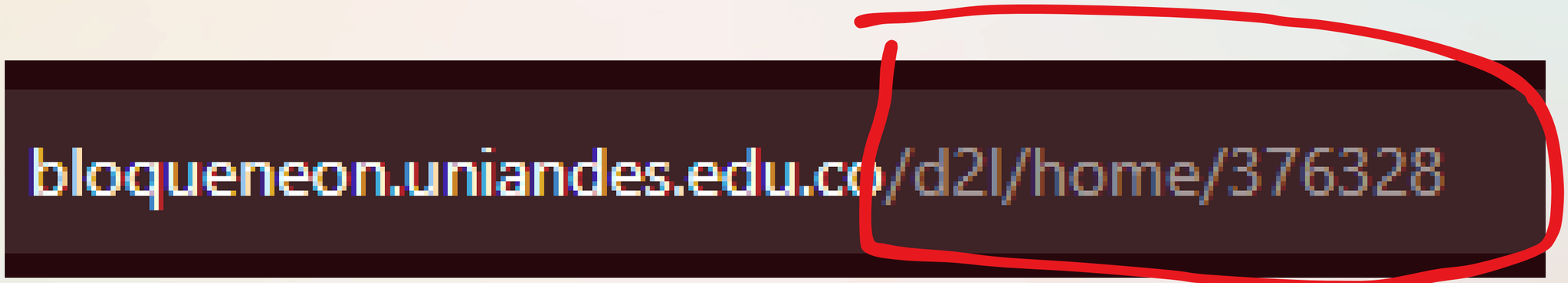
Pero en el navegador sale el siguiente link:



¿Es diferente a cuando entramos a la página principal?

Peticiones Web

Lo que sale después del link de cada / nos indica una ubicación dentro de la página



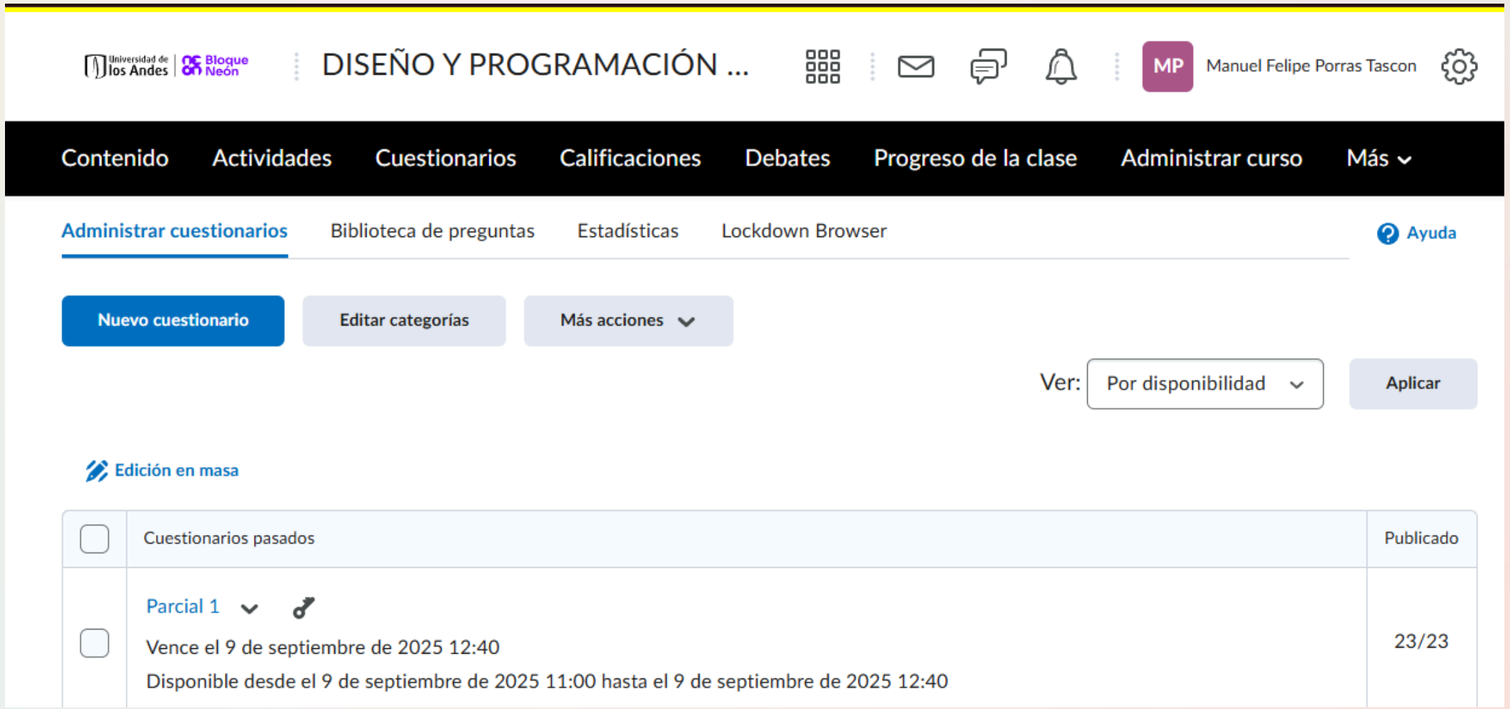
En este caso nos indica que entramos a la vista de un curso con código 376328 == El curso de bloqueneon de nuestra sección de DPOO.

Peticiones Web

¿Ahora qué significa lo siguiente?

Le di click a cuestionarios de mi curso de DPOO

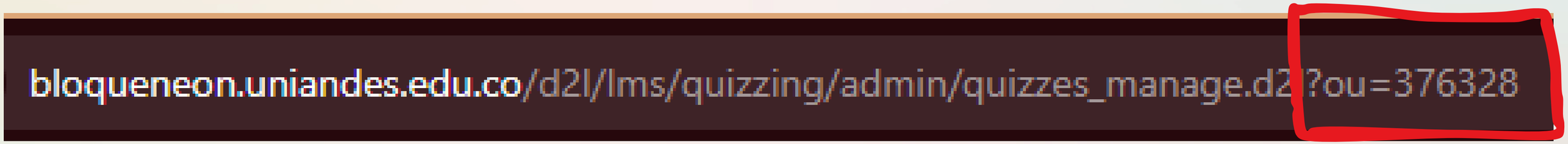
bloqueneon.uniandes.edu.co/d2l/lms/quizzing/admin/quizzes_manage.d2l?ou=376328



Peticiones Web

¿Ahora qué significa lo siguiente?

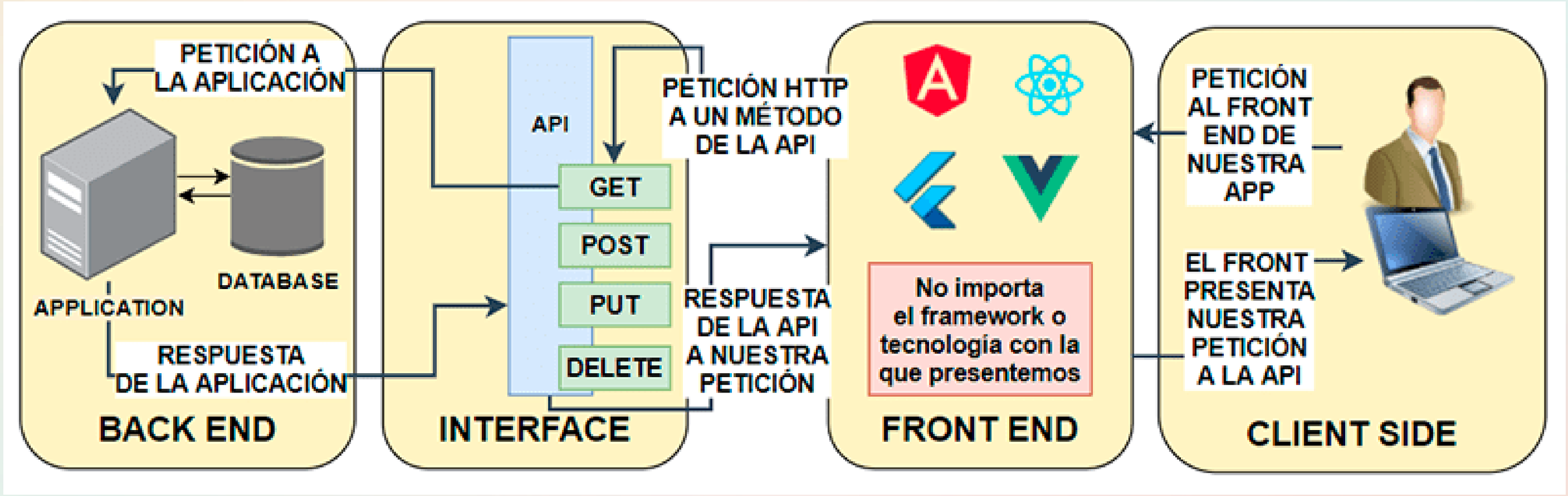
Le di click a cuestionarios de mi curso de DPOO



En esta ocasión nos dice que vamos a la zona de administrador de cuestionarios del curso con ID 376328.

El ? nos indica la presencia de un parámetro en una petición

Peticiones API



Peticiones API

HTTP Request

Request URL

http://jarroba.com/

Request Method

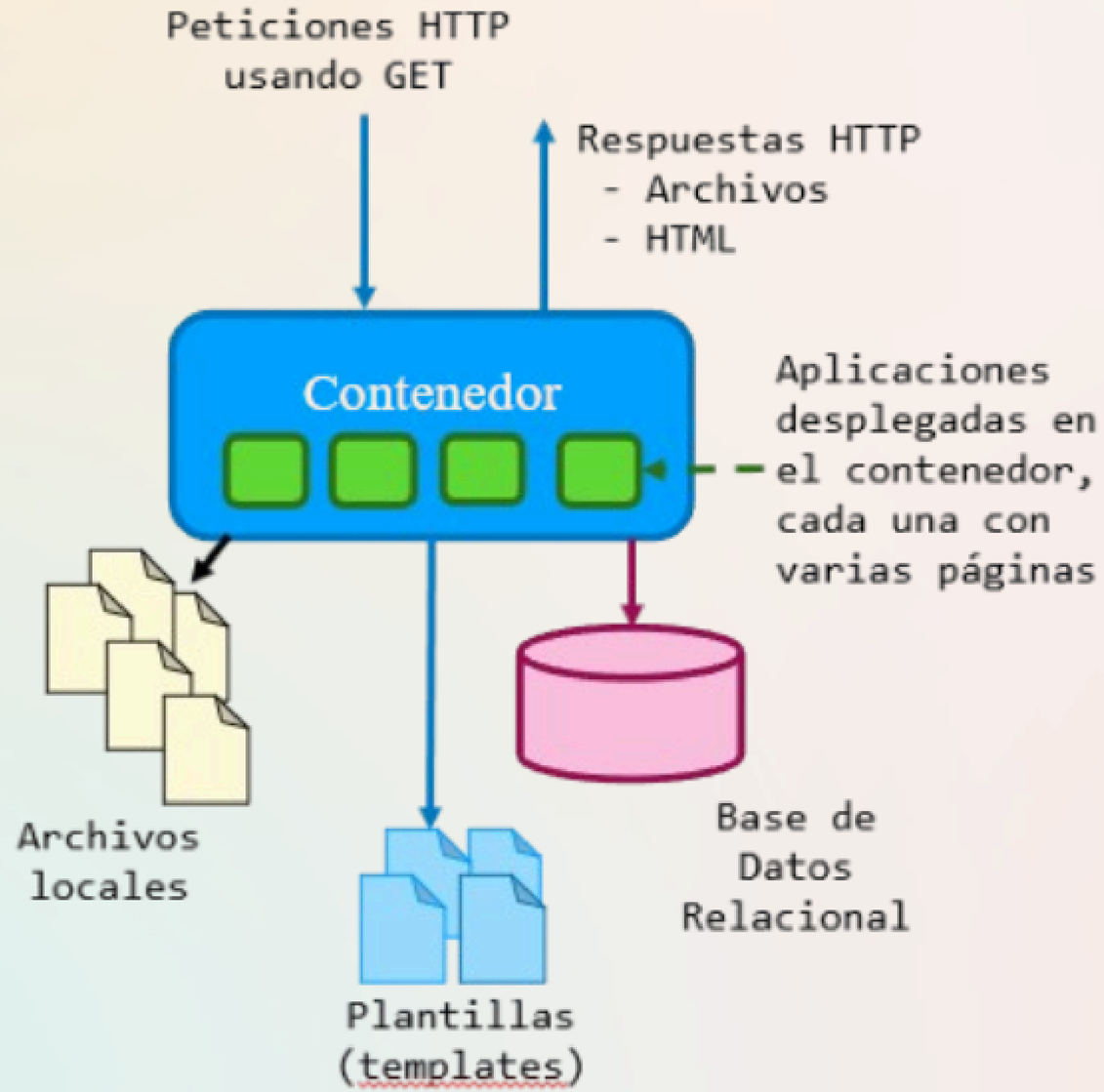
GET

Request Headers

Accept: text/html, image/png
Accept-Encoding: gzip, deflate, sdch
Accept-Language: es-ES, en-US
Host: jarroba.com
Referer: http://desde_donde_vengo/
User-Agent: (Windows NT 6.3; WOW64) Chrome/45.3

Ahora si, volvamos al ejemplo con el que vamos a trabajar

Contenedor de aplicaciones web



Cada aplicación va a estar compuesta de varias páginas, y el contenedor tiene que ser capaz de enrutar cada solicitud a la aplicación y página correcta.

Contenedor de aplicaciones web

Condiciones a tener en cuenta

El contenedor debe crear sesiones para cada petición nueva y debe ser posible almacenar información dentro de esa sesión.

Para generar los archivos HTML, el contenedor debe tener un sistema de plantillas (templates)

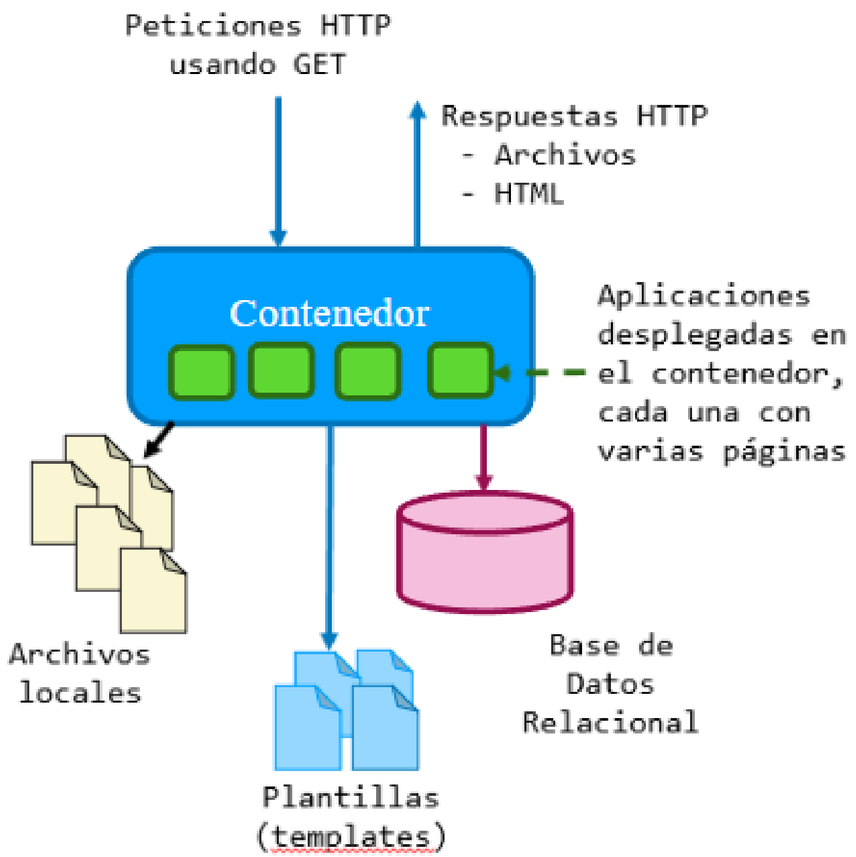
El contenedor debe permitir que para la construcción de las páginas se hagan consultas sobre una base de datos relacional.

El contenedor también será capaz de entregar archivos estáticos (ej. “inventario.pdf”)

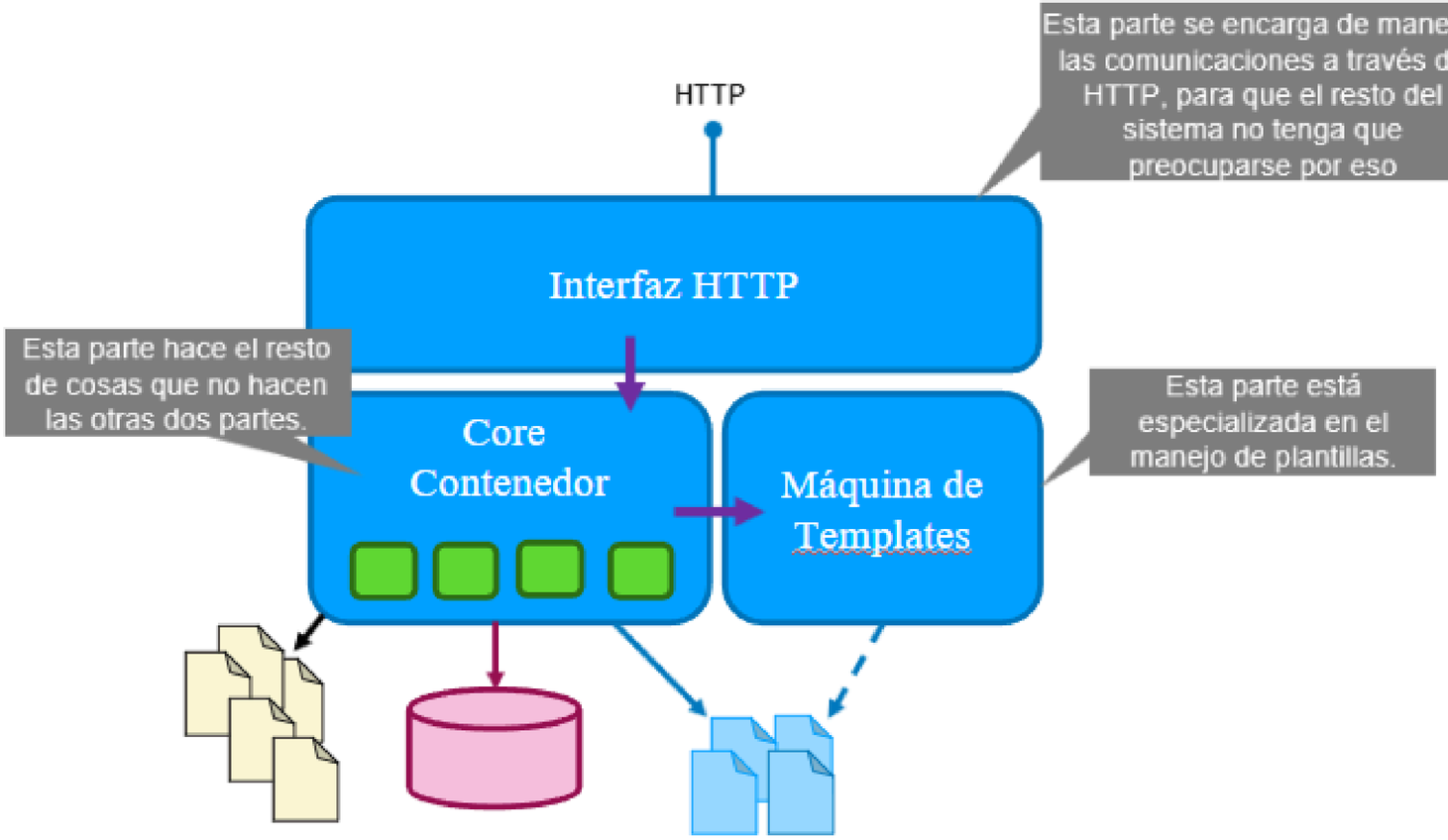
Mostraremos una posible solución de muchas

Descomposición del problema

Diagrama de contexto



Primera descomposición de la solución



En esta primera descomposición del problema estamos intentando simplificar el trabajo y no tener que solucionarlo todo al tiempo. Los dos componentes que se separaron tienen que ver con aspectos específicos que tienen una complejidad relativamente alta. Es mejor separarlos y ocuparnos de ellos por aparte, sin complicar otros aspectos de nuestro sistema.

Responsibility-Driven Design

Instructor Manuel Porras
mf.porras@uniandes.edu.co

Responsibility-Driven Design

Es una metodología propuesta por Wirfs-Brock y McKean para diseñar software.
Está basada en 3 conceptos principales:

Roles

Responsabilidades

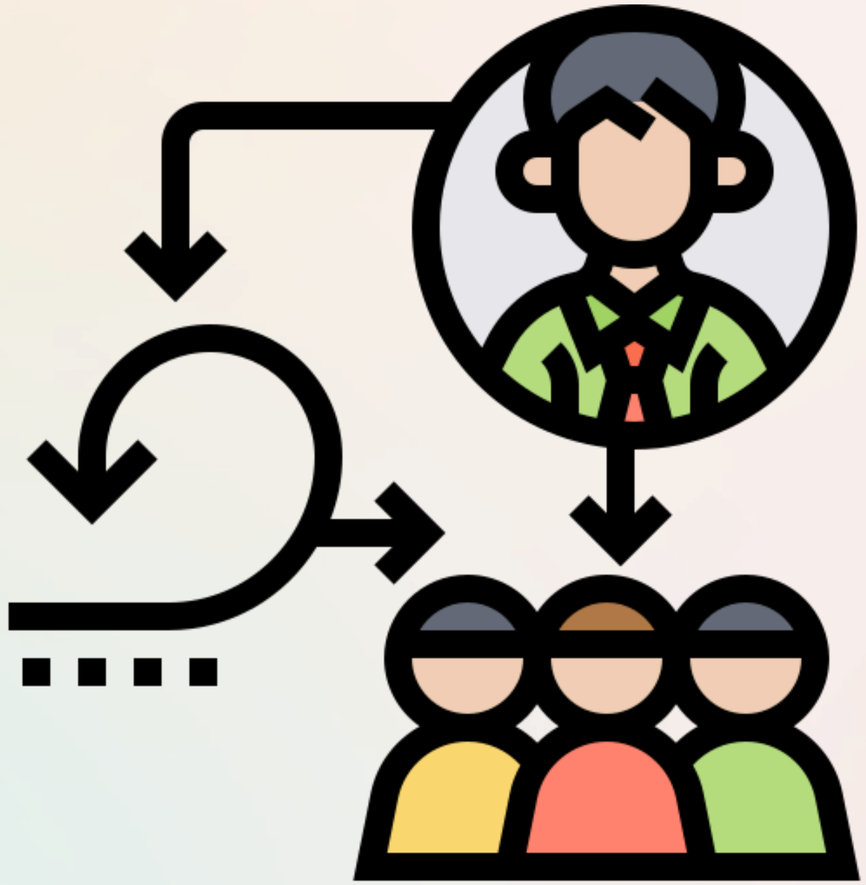
Colaboraciones

No es la única metodología que existe, pero es una buena metodología y la utilizaremos de acá en adelante.

Roles > Responsabilidades > Colaboraciones

Objetos y roles

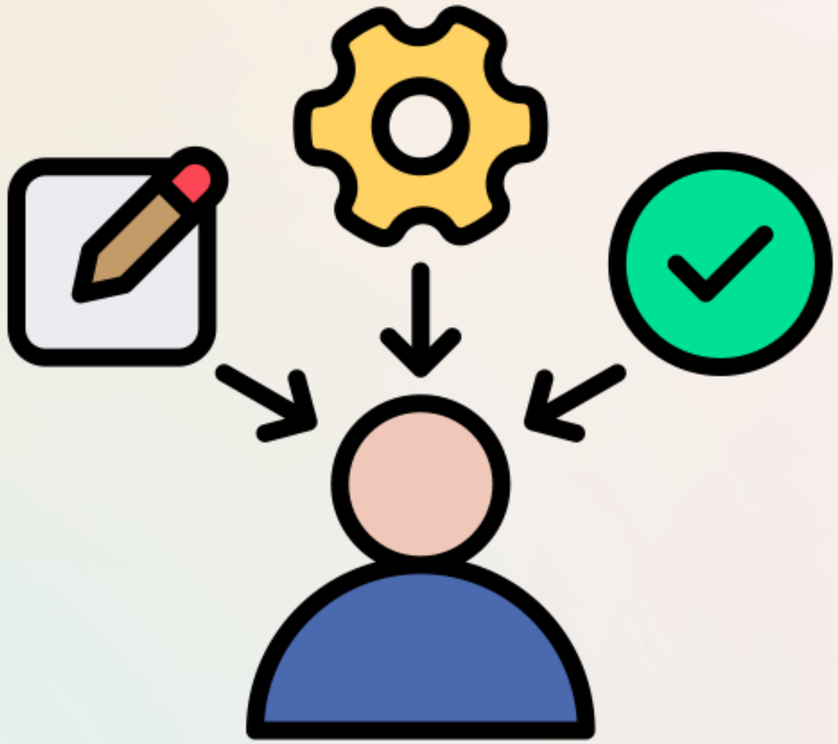
¿Quiénes participan en la solución?



Dentro de una aplicación, cada objeto juega uno o varios roles, y un rol puede verse como un conjunto de responsabilidades que alguien debe asumir.

Responsabilidades

¿Qué responsabilidades tienen los participantes?



Las responsabilidades se asignan a un rol particular, pero puede ser que los objetos que asuman el rol requieran de la ayuda de otros objetos para poder cumplir con sus responsabilidades.

Colaboraciones

¿Cómo interactúan los participantes para cumplir con sus responsabilidades?

Cuando se diseñan colaboraciones los objetos dejan de considerarse elementos independientes y empiezan a estudiarse como aliados.

Además, existen unos estilos de control

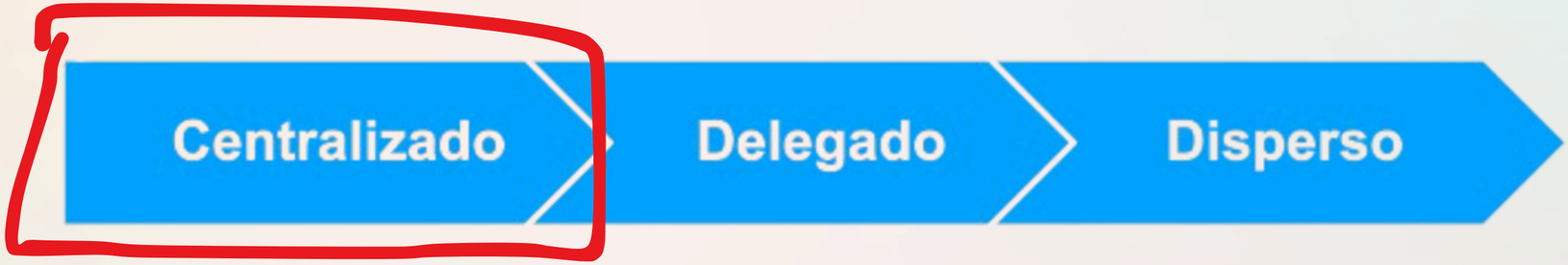
Estilos de control

¿Quién tiene la capacidad para tomar decisiones ?,
¿Quién tiene el acceso a la información para tomar esas decisiones?,
y ¿Quién tiene el contacto de los otros elementos para poder pedirles que realicen otras operaciones.?

El estilo de control hace referencia a la forma en la que esas responsabilidades identificadas se distribuyen.

Estilos de control

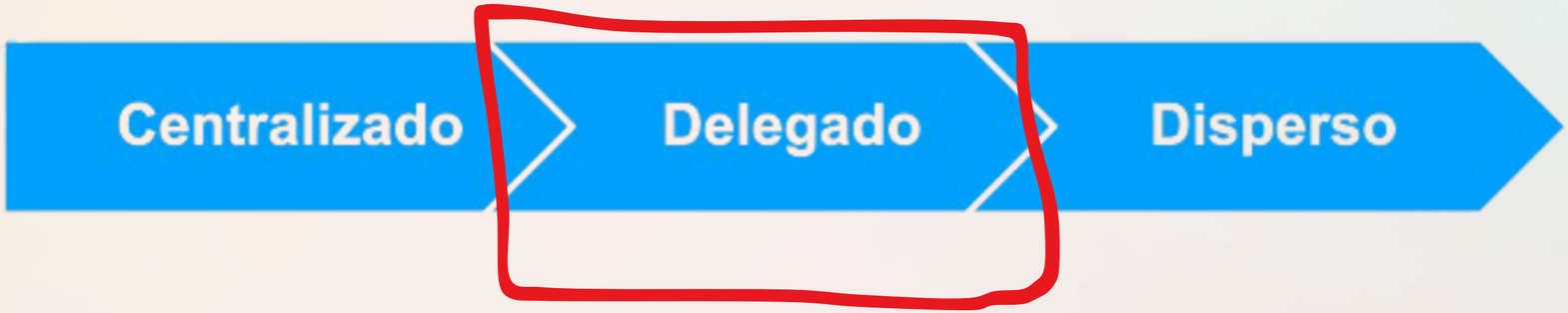
Existen 3 tipos de estilos de control



Si el estilo de control es **centralizado**, la toma de decisiones se concentra en pocos elementos (típicamente, controladores).

Estilos de control

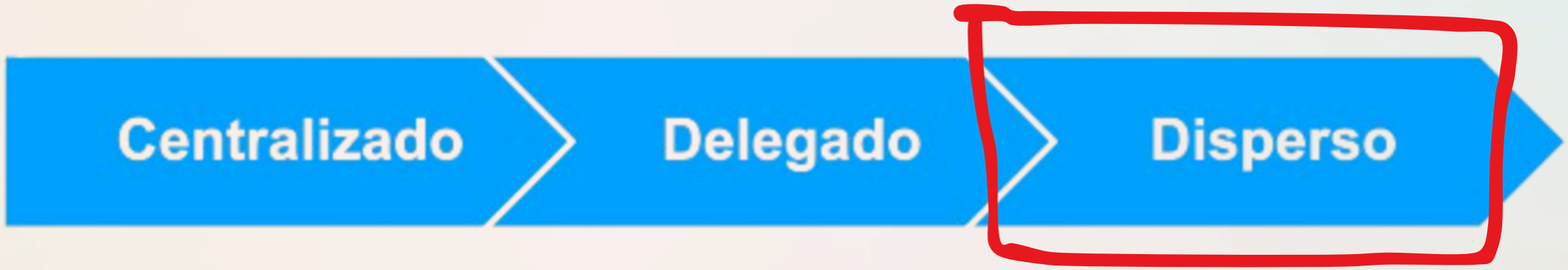
Existen 3 tipos de estilos de control



Si el estilo de control es delegado, la toma de decisiones se divide entre algunos elementos centrales y otros elementos más pequeños. En general los primeros terminan actuando como coordinadores.

Estilos de control

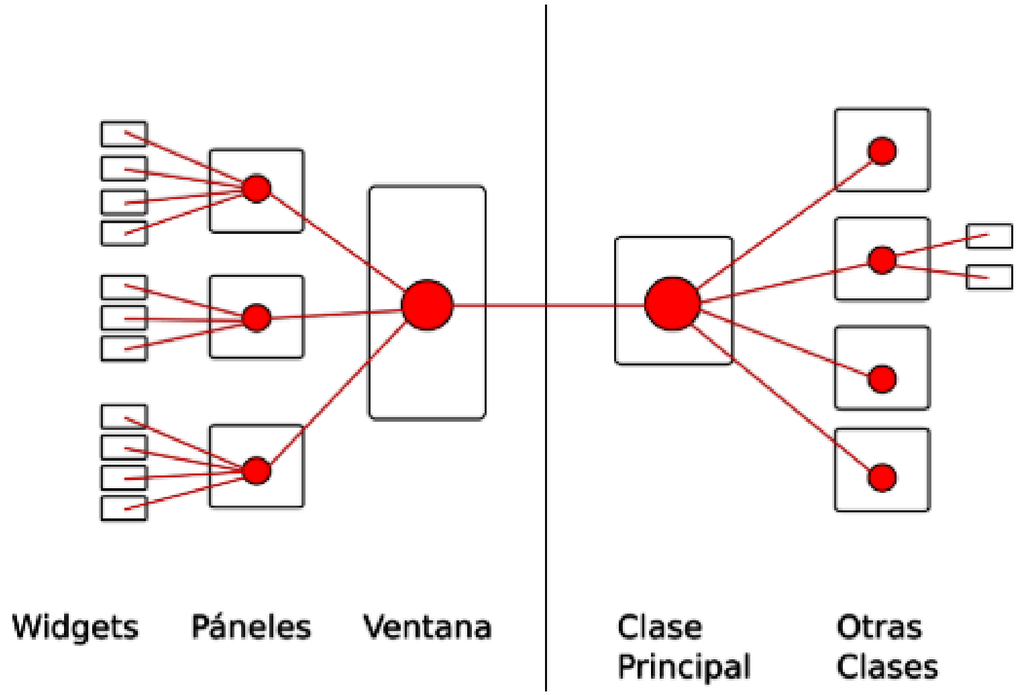
Existen 3 tipos de estilos de control



Si el estilo de control es disperso, la toma de decisiones se distribuye entre muchos elementos. En este caso las decisiones tienden a ser más pequeños y se vuelve difícil ubicar los centros de control de la aplicación.

Ejemplo de Control Centralizado

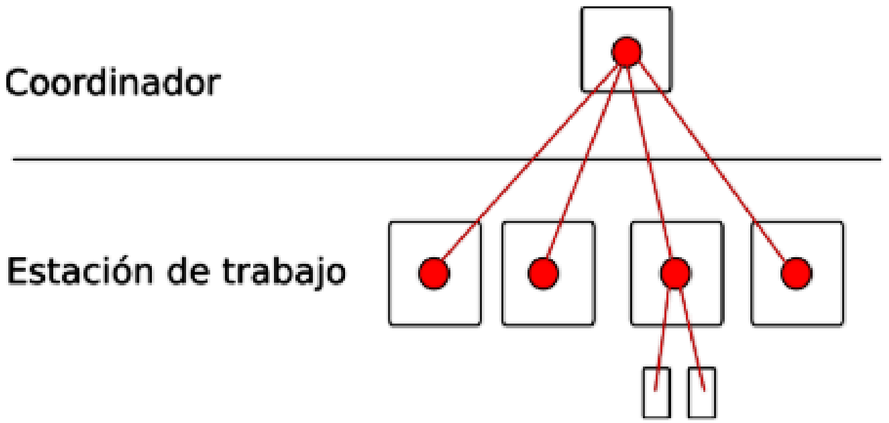
- Esta es una representación simplificada de la arquitectura típica de las aplicaciones desarrolladas en los antiguos cursos de APO1 y APO2: si usted entra a http://cupi2.uniandes.edu.co y revisa el código fuente de los programas publicados, en la mayoría de los casos se encontrará con esta misma estructura.



- Esto significa que tanto en el paquete de la interfaz como en el paquete del mundo generalmente había una clase concentrando la mayor parte de la lógica (la inteligencia del

Ejemplo de Control Delegado

- En una planta ensambladora de carros automatizada se puede ver un estilo de control delegado:
  - Hay unos elementos principales que se encargan de hacer un control de alto nivel, moviendo los proto-carros entre las diferentes estaciones de la cadena de ensamble.
  - Cada una de las estaciones tiene que hacer un conjunto de tareas pero tiene total control sobre las actividades que realiza. Estas estaciones podrían tener también sub-estaciones que se controlarían de la misma forma.
  - En este modelo, los elementos que hacen control de alto nivel en realidad tienen el rol de coordinadores.



Elementos implementables

La identificación y diseño de roles, responsabilidades y colaboraciones puede hacerse a cualquier nivel dentro del proceso de diseño:

En el nivel de la aplicación completa, solo hay un rol (la aplicación),las responsabilidades son todos los requerimientos, y las colaboraciones ocurren con los elementos externos (actores, otros sistemas).

Elementos implementables

La identificación y diseño de roles, responsabilidades y colaboraciones puede hacerse a cualquier nivel dentro del proceso de diseño:

En el nivel más bajo, los roles serán asumidos por objetos, que tendrán que implementar responsabilidades en sus métodos, y colaborarán con otros objetos.

Elementos implementables

La identificación y diseño de roles, responsabilidades y colaboraciones puede hacerse a cualquier nivel dentro del proceso de diseño:

Cuando llegamos al nivel más bajo es cuando finalmente diseñamos en términos de los conceptos de programación orientada a objetos (clases, métodos, atributos, interfaces, etc.).

El resumen

- ✓ Una aplicación → un conjunto de objetos que interactúan
- ✓ Un objeto → la implementación de uno o más roles
- ✓ Un rol → un conjunto de responsabilidades relacionadas
- ✓ Una responsabilidad → la obligación de hacer una tarea o de tener información
- ✓ Una colaboración → la interacción de objetos o roles
- ✓ Un contrato → el acuerdo que describe los términos de una colaboración

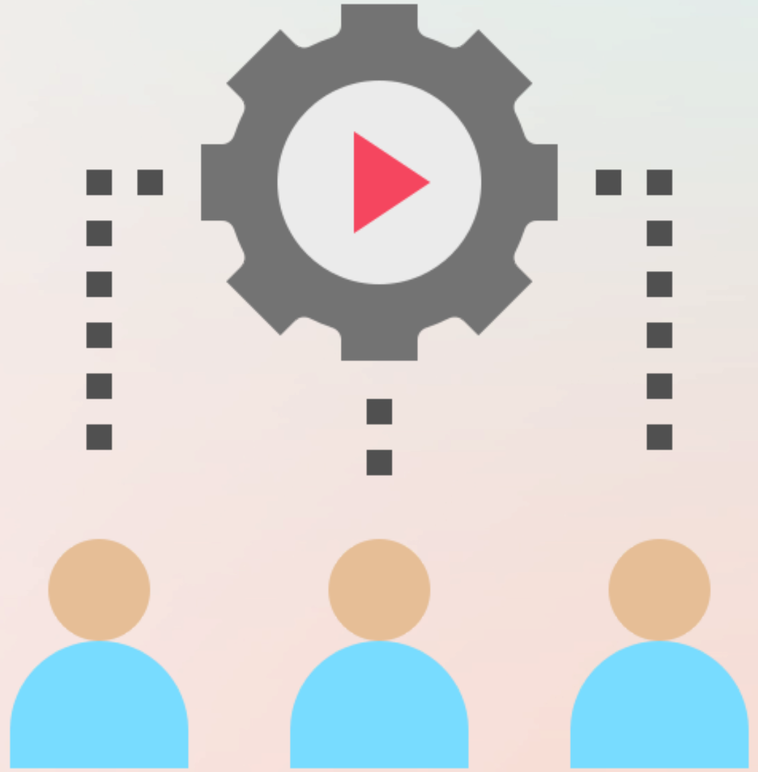
Roles > Responsabilidades > Colaboraciones

Ahora, ¿Cómo sabemos qué es un rol, una colaboración y un comportamiento?

Objetos y roles: quién participa en la solución

Componentes / Roles / Clases … **según el nivel** en el que estamos diseñando.
Los roles son abstractos en principio, pero más adelante se convertirán en elementos concretos.

Iniciaremos buscando componentes candidatos que iremos refinando: algunos desaparecerán, otros se mezclarán y otros nuevos aparecerán más adelante.



Estereotipos

Los estereotipos son roles comunes y muy simplificados, que sirven para iniciar la discusión sobre las responsabilidades de un elemento, y pueden ser útiles para identificar los candidatos.

Los siguientes son 6 estereotipos que se encuentran muy frecuentemente:

1. Information holder: mantiene y entrega información
2. Structurer: mantiene las relaciones entre objetos y provee información sobre esas relaciones
3. Service provider: hace algún trabajo para los demás (ofrece un servicio complejo)
4. Coordinator: su principal responsabilidad es delegar tareas a otros componentes
5. Controller: toma decisiones importantes y controla las acciones de otros componentes
6. Interfacer: transforma información y peticiones entre diferentes partes de un sistema

ROLES Y ESTEREOTIPOS EN PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

Los estereotipos son roles comunes y muy simplificados, que sirven para iniciar la discusión sobre las **responsabilidades** de un **elemento**, y pueden ser útiles para identificar los candidatos.

1. Portador de Información (INFORMATION HOLDER)

Un portador de información de objetos, información de datos para identificar candidatos.

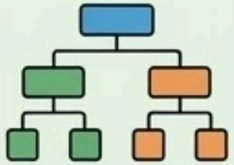
```
class Usuarto {
  String nombre;
  ...
  getNombre()
}
```

Ejemplo de archivo físico.

2. Estructurador (STRUCTURER)

Un estructurador estructurado de las organizaciones y para elementos de un organigrama de negocio.

```
class Organizacion {
  List<Departamento> depts;
  ...
}
```



3. Proveedor de Servicios (SERVICE PROVIDER)

Un proveedor de servicios ofrece u servicio de servidor, ofrece un servicio de autenticación.

```
class ProcesadorPagos {
  void procesar(monto)
  ...
}
```

Un servidor ofrecnd un servicio de autenticación.

4. Coordinador (COORDINATOR)

Un director de orquesta que coordina en el manejo del proyecto.

```
class PlanificadorViajes {
  Vuelo.reservar();
  Hotel.reservar();
}
```

5. Controlador (CONTROLLER)

Un controlador de controles de tráfico aéreo.

```
class MotorJuego {
  update();
  handleInput();
}
```

6. Interfaz (INTERFACER)

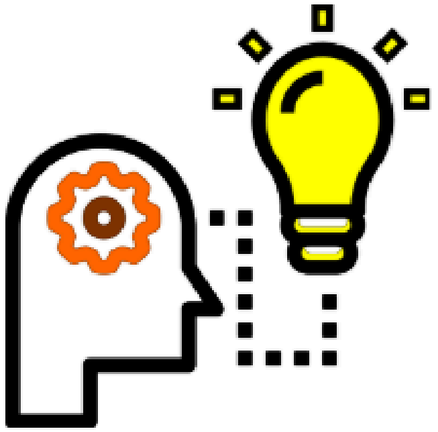
Una interfaz de adaptador de viaje universal es un adaptador de enchufe universal.

```
class AdaptadorClimaApi {
  DatosClima convertir(vjosDatos)
}
```

Imagen generada con IA puede contener errores*

Técnicas para descubrir componentes candidatos

Podemos encontrar tanto elementos del dominio (ej, Documento, Competidor, Resultado) como elementos que sólo tienen sentido dentro del sistema - Application Specific Objects (ej. TransactionService, ApplicationInitializer).



Las siguientes son perspectivas que podríamos asumir para descubrir candidatos:

- El dominio del problema y los elementos con los cuales trabaja el sistema.
- Las responsabilidades del sistema (para buscar elementos que cumplan con esas responsabilidades o con partes de esas responsabilidades).
- Elementos conectados o afectados por nuestro sistema (digitales o reales).
- Información que fluya por el sistema.
- Actividades de toma de decisiones, control y coordinación.
- Estructuras y grupos de objetos.
- Representación de objetos del mundo real que deban aparecer dentro del sistema.

Descripción de candidatos

Cada candidato debe tener:

- Un buen nombre
- Una descripción
  - Presentar la intención por la cual el elemento fue escogido entre los candidatos
- Un estereotipo
  - Puede estar basado en los que mencionamos antes

Qué es un buen nombre

- Si es genérico (Loader) debe tener más información
- No poner demasiados detalles
- Mostrar claramente que elementos trabajan (Workers, ServiceProvider) y cuáles contienen información
- No seleccionar nombres muy restrictivos
- Usar nombres que coincidan con el ciclo de vida de los objetos
- Usar los nombres sólo una vez
- Evitar conflictos con adjetivos y sinónimos
- Usar nombres claros

Identificar objetos / roles candidatos

1. Elementos relacionados con en el dominio del problema

Petición
Parámetro
Respuesta
Sesión
Página
Ruta
Cookie
Query
Archivo
Plantilla

2. Elementos relacionados con lo que hace el sistema

Manejador Sesiones
Buscador Archivos
Enrutador
Conector DB

3. Cosas que fluyen dentro del sistema

Conexión DB
Plantilla Llena
Resultado Query

4. Agrupadores

Aplicación

Estos son los primeros elementos candidatos que se identificaron para el caso de estudio, agrupados de acuerdo a la perspectiva que se utilizó para encontrarlos. Los grupos no quieren decir que, por ejemplo, un Query no pueda ser algo que fluya dentro del sistema – los grupos solamente muestran en qué momento se encontró cada elemento.

Asignar estereotipos

1. Elementos relacionados con en el dominio del problema

Petición
Parámetro
Ruta
Cookie
Página
<<controller>>
Sesión
Plantilla
Archivo
Respuesta
Query

2. Elementos relacionados con lo que hace el sistema

Manejador Sesiones
<<structurer>>
Buscador Archivos
<<structurer>>
Enrutador
<<coordinator>>
Conector DB
<<service provider>>

3. Cosas que fluyen dentro del sistema

Conexión DB
<<interfacer>>
Plantilla Llena
Resultado Query

4. Agrupadores

Aplicación
<<structurer>>

Estereotipos

1. Information Holder
2. Structurer
3. Service provider
4. Coordinator
5. Controller
6. Interfacer

Todos los elementos que NO tienen un estereotipo marcado son <<information holder>>
La asignación inicial de estereotipos no es ni precisa ni definitiva, pero nos empieza a dar ideas para la asignación de responsabilidades que haremos más tarde.

Asignar estereotipos

La asignación de estereotipos que se mostró tiene varias consecuencias muy importantes:

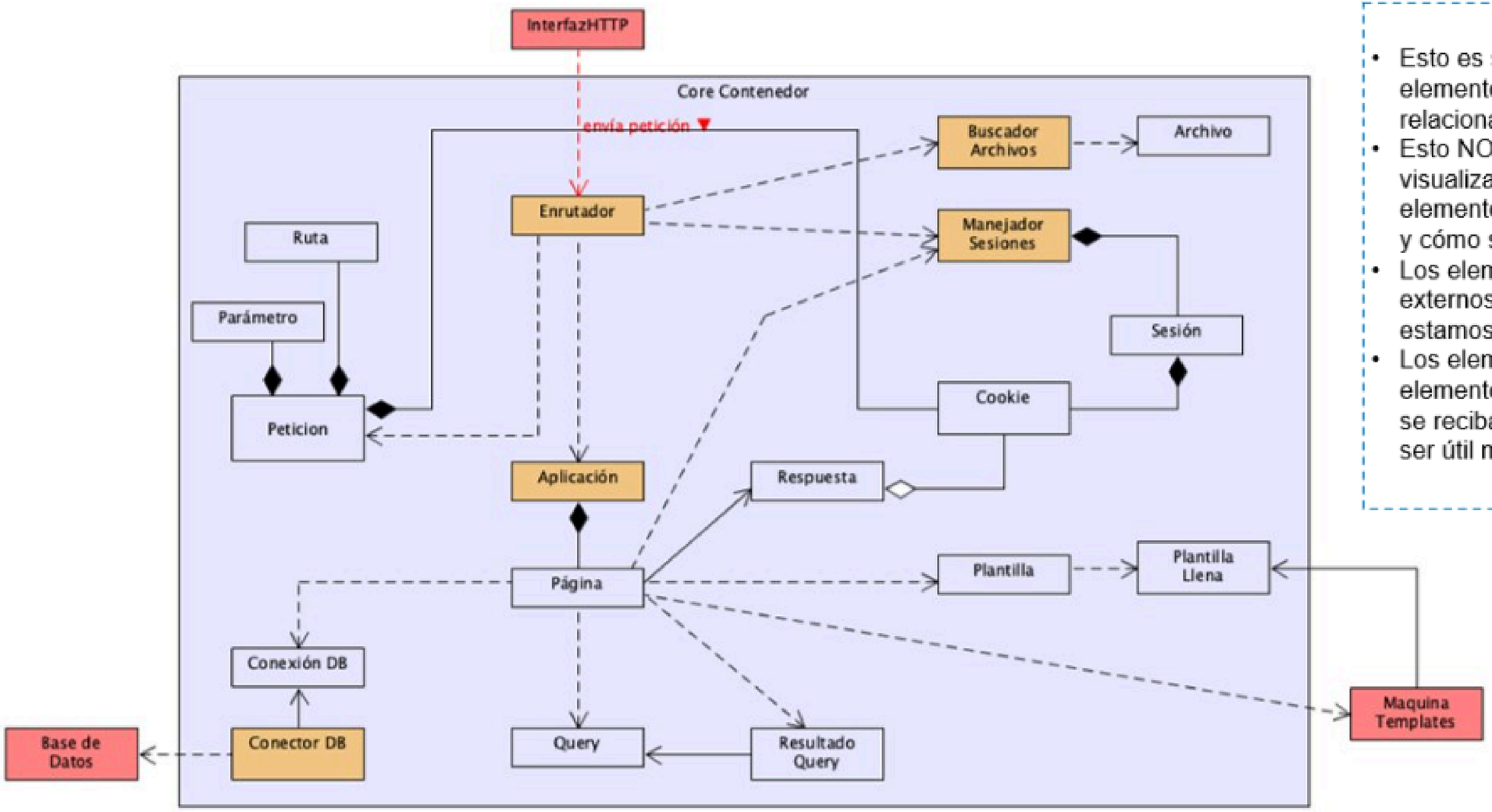
- Si las Páginas son **controladores**, significa que ahí se concentrará mucha de la lógica del sistema y que las páginas tendrán que interactuar con otros elementos para lograr sus objetivos.
- Si el Enrutador es un **coordinador**, significa que posiblemente no tendrá una lógica complicada y que delegará en otros realizar las tareas complejas.
- Si el BuscadorArchivos es un **estructurador**, significa que no tendrá una lógica compleja y que su rol será principalmente organizar los archivos para permitir que se acceda a ellos.
- Si la Aplicación es un **estructurador**, significa que tampoco tendrá una lógica compleja; probablemente sirva sólo para mantener organizadas unas páginas.
- Si ConexionDB es un **interfacer**, significa que su rol principal será mediar en la interacción con un elemento externo (en este caso, la base de datos).

¿ Y si agrupamos
todos esos roles?

Después de tener los elementos, la siguiente tarea es empezar a buscar relaciones:

- Buscar clústeres.
- Buscar elementos con características comunes o comportamientos cercanos.
- Buscar elementos que compartan abstracciones.

Agrupación/Organización de elementos



- Esto es sólo un primer intento de agrupar los elementos y de entender cómo podrían relacionarse entre ellos.
- Esto NO es un diagrama de clases, es una visualización informal para entender qué elementos candidatos tenemos hasta el momento y cómo se podrían relacionar.
- Los elementos marcados en rojo son elementos externos al Core Contenedor (el componente que estamos diseñando).
- Los elementos marcados en naranja son elementos que no dependen de cada petición que se reciba (siempre deberían existir) … esto podría ser útil más adelante.